

AI Usage Log: Sprint 1

Task: Dashboard frontend (resume upload UI)

Purpose of AI Use: Code generation and UI/UX design for the Sprint 1 dashboard.

Chat Link or Prompt/Response: I'm doing the frontend dashboard for CareerConnect in React/TypeScript. My teammates already worked on login and the resume backend. Can you help me generate Dashboard.tsx and the dashboard styles in index.css around those existing parts? Use the User and Resume types and the resume helpers from api.ts rather than making another data shape or new routes. The dashboard receives the logged-in user, their token, and a logout handler. I want the UI to show the resumes saved for that user when the page opens, and to use the backend response after an upload so the list matches what was really saved.

For the design, I want a cobalt-blue sidebar on the left and a clean white main area like this: CareerConnect at the top with Career in white and Connect in yellow, a large rounded profile box with the user's initials, then their name, role, and email. Put Resumes and Log out near the bottom. In the main area, have a big 'Resumes' title, 'PDF or DOCX, up to 5 MB' under it, and a rounded blue 'Upload resume' button on the right. Below it should be a wide dashed drop zone saying 'Drop a resume here or browse your files'. The button, browse link, and a plus card should open the same file picker. Show 'No resumes yet' when the list is empty, then show each uploaded resume as a card with its filename, file type, date, and Download/Delete actions. A simple paper-looking placeholder is fine because we don't have actual thumbnails.

Please support drag and drop too. Check PDF/DOCX and the 5 MB limit before sending a file, show a useful error if it fails, and disable the upload controls while it is uploading. Keep the file handling in one function so picking and dropping work the same way. Only job seekers need this resume screen; a short placeholder is enough for recruiters. Make it responsive on phones, and add a dark color scheme without changing the layout. Only put dashboard CSS after the `/* CareerConnect dashboard */` marker, because the CSS above it belongs to my teammate. Give me usable React code and CSS, and briefly explain the important parts so I can review them.

AI-Suggested Content:

For Dashboard.tsx, use the logged-in user, token and logout callback as props. The sidebar can show initials from the user's name, their role and email, with Resumes and Log out at the bottom. A job seeker sees the resumes page from your design. A recruiter gets a short placeholder for future tools.

The Upload resume button, browse link and plus card open one hidden file input. Drag and drop goes through the same `addFile()` function. Check the `.pdf` or `.docx` extension and the 5 MB limit before calling `uploadResume(file, token)`. Show a clear error if the file is invalid or the request fails. While an upload is in progress, show 'Uploading...' and prevent another upload. Add the Resume returned by the server only after the upload succeeds.

Load existing files with `listMyResumes(token)` when the job seeker opens the dashboard. For each card, show the filename, type and upload date, with Download and Delete actions using the existing `api.ts` helpers. Keep a failed delete on screen and show an error. The paper-style `SheetSketch` is a placeholder, since the Resume data does not include a thumbnail.

For index.css, add the dashboard styles only after the `/* CareerConnect dashboard */` marker. The CSS uses a cobalt sidebar and white main area in light mode, darker colors from `prefers-color-scheme` in dark mode, dashed upload and add areas, and a two-column card grid on a phone.

Validation: Previewed designs before adding code. Ran the UI locally; tested PDF/DOCX, wrong file types, files over 5 MB, and a phone-size screen. Checked that the teammate's CSS above the marker was unchanged.

Decision: Modified. I rejected the first complicated version and another design I disliked, chose the cobalt layout and dark colors, and changed the logo. The proposed API calls later caused an auth error, so I removed them from my part as described in the backend connection task.

Reflection: Building and testing the dashboard helped me learn React, especially how props, state, file inputs, and event handlers work together for uploads and drag and drop.

Responsible Person: Essam

Task: Connecting the dashboard to the backend

Purpose of AI Use: Debugging the proposed resume API integration and token error.

Chat Link or Prompt/Response: I want to know whether uploading a CV from my dashboard actually saves it to the database. The resume routes are in the backend, and I have api.ts and App.tsx. Can you point out the lines that would load and upload resumes, and show where the login token comes from? I tried connecting it, but I still get 'Token is invalid or expired' right after a fresh login. Please explain why this error appeared when my old UI worked, and what I should check next. If the problem is in auth, I'd rather keep my dashboard focused on the UI and let the teammate who owns that part connect it.

AI-Suggested Content:

Your original dashboard only added files to its browser-side list. Seeing a row in a resumes table does not prove this version of the UI wrote it. To connect the UI, it would need to request the user's resumes when the dashboard loads and send the selected file to the upload route. Both calls need the token from the login flow. The proposed endpoints were /api/resume/mine and /api/resume/upload.

The new fetch and upload calls are the lines that introduced the token error: before those calls, your dashboard never asked the backend to authenticate anything. App.tsx also matters because the login token may be read from sessionStorage as well as localStorage. Checking both locations can fix a missing-token case, but it does not explain a backend rejecting a token that was issued moments earlier.

Since the error continued after a fresh login, the frontend change alone did not resolve it. The backend auth code, token signing/decoding, and the login response need to be checked together by the teammate who owns that path. You can remove the proposed API calls from your dashboard for this sprint and retain the same visual UI. Tell the team what routes you tried and quote the exact error so they have a reproducible starting point.

Validation: Tested locally after login and again after a fresh login. The error remained. Asked for the exact lines that contacted the backend.

Decision: Rejected the backend integration in my part. I removed those calls and left the dashboard UI ready for the teammate responsible for auth/backend; a teammate later connected it.

Reflection: I understood which parts of the dashboard made API requests and why the browser-only UI had not shown the token error.

Responsible Person: Essam

Task: Git workflow, pull request, and team messages

Purpose of AI Use: Working through a branch/merge conflict and preparing the dashboard PR.

Chat Link or Prompt/Response: My teammate asked me to commit the dashboard so the team can review it in a PR. My changes are still local on main, but there is already an Essam branch on GitHub. How do I move just Dashboard.tsx and its CSS to that branch without losing anything? I also need to bring in the latest main. Git is giving me branch and merge errors, and Dashboard.tsx conflicts with a resume section a teammate added. Walk me through the simplest way to resolve that and check the final diff. Then help me write a short PR description and team message that say what my UI includes and that backend integration is separate.

AI-Suggested Content:

First inspect git status and keep a copy of the two edited files. Use the existing Essam branch rather than creating another one. If Git refuses to switch while the files are modified, stash or otherwise save the local changes, switch to that branch, and restore them there. Commit only the dashboard component and its CSS so the PR has a clear scope.

Bring in the latest main before the PR if the branch is behind. A conflict in Dashboard.tsx means Git found overlapping edits; it cannot decide which resume section should survive. Review both versions rather than assuming either one is correct. If you keep your dashboard version to get the work into a reviewable PR, describe the teammate's overlapping component in the PR so the team can reconcile it there. Confirm the final diff and git status before pushing.

A short PR description can say: 'Added the Sprint 1 dashboard UI for resume upload/profile management. The dashboard section of index.css is included; shared CSS above it was left alone. The UI accepts PDF/DOCX and shows resume cards. Backend integration is being handled separately.' Add the local checks you actually ran. Then send the team a plain note with the PR link and ask them to review the dashboard and the overlap in Dashboard.tsx.

Validation: Checked git status at each step and confirmed the PR on GitHub. Checked that the description matched the actual two-file change.

Decision: Modified. I used the existing branch, kept my dashboard version when the conflict became complicated, and called out the overlap to the team. I shortened the suggested PR and team messages.

Reflection: I learned why syncing main matters before a PR and why committing directly on main causes trouble for the team.

Responsible Person: Essam

Task: Team Process Definition

Purpose of AI Use: Help drafting and checking the team's branching, PR, review, Ready, and Done rules.

Chat Link or Prompt/Response: I need to add the branching strategy, PR process, code review process, Definition of Ready, and Definition of Done to TeamProcess.md. Let's do one section at a time so I can check each part. I'm attaching screenshots of our actual branches. The feature/ and fix/ names in your first suggestion don't match our repo; we use main and have agreed to move from member branches to feature branches. Help me decide which rules belong in each section and which ones are course requirements versus general practice. For example, is deleting a branch after merging required, and should creating a feature branch be in the branching section or PR steps? Judge those points yourself. Once I paste the full draft, check that the sections agree with each other and with our real workflow.

AI-Suggested Content:

Branching strategy. Keep main as the shared branch and make new work on a feature branch from the latest main. Describe branch names using the convention your team actually adopts; do not put feature/ or fix/ prefixes in the document as an existing rule when your repository does not use them. The step for creating a branch belongs here, before the PR process. Deleting a branch after merge is cleanup the team may choose, not a condition for the code to work.

Pull request process. Commit the feature work, sync it with main, resolve conflicts, and open a PR with a clear summary and the tests run. Ask for the required review approval before merging into main. This section should start once work is ready for review; repeating 'create a feature branch' here makes the order confusing. Make sure every mention of member branches is updated if the team has moved to feature branches.

Code review. The reviewer should read the changed files for scope, secrets, and obvious issues, run or test the branch when practical, and leave a specific review comment. Approval should mean the reviewer checked the work, not just clicked the button. For Ready and Done, use the sprint instructions as requirements and label extra Agile checklist ideas as team choices rather than course rules.

When the full draft was checked, three consistency problems stood out: one workflow sentence still said dev instead of main, the branching strategy needed a sentence to connect its steps, and the wording about who assigns tasks did not agree across sections. 'Start a feature branch from the latest main' means update your local main first, then branch from that current state; it does not mean starting from an old copy of main.

Validation: Compared the draft with the actual branch screenshot, repository workflow, and sprint requirements. Asked where the Ready points came from and pasted the entire document for a consistency check.

Decision: Modified. I rejected branch prefixes that did not fit the repository, reconciled feature versus member branches, moved branch creation to the branching section, questioned branch deletion, and corrected dev to main.

Reflection: Working section by section let me challenge the draft against our real workflow. The full-document review caught mistakes that were hard to see in isolated sections.

Responsible Person: Essam

Task: Reviewing a teammate's pull request (#74)

Purpose of AI Use: Applying the team's code review process to a teammate's PR.

Chat Link or Prompt/Response: My teammate's PR #74 is waiting for my review. Using the checklist in our Team Process document, what should I check before approving it? I checked out the branch and tested registration and login, uploaded a resume and confirmed it was still there after refresh, and looked over the dashboard. Those parts worked locally. Should I approve the PR now, and would it help to leave a short comment saying exactly what I tested?

AI-Suggested Content:

Review the changed files first to see whether the PR includes only the intended work and whether anything sensitive or unrelated was added. Then run the branch locally and test the user path, not just the individual components. Your registration/login, resume upload, persistence after refresh, and dashboard checks cover the core behavior of this PR.

If those checks passed and you did not find a blocking problem, submit an approval. A short comment is useful because it tells the author what you actually verified: 'Tested registration/login, resume upload and persistence after refresh, and the dashboard locally. Everything worked for me.' If you saw a limitation, mention it separately rather than implying it was tested.

Validation: Checked out and tested the branch myself: login/register, resume upload and persistence after refresh, and the dashboard design.

Decision: Accepted the review procedure. I approved PR #74 after my local tests.

Reflection: The local test is what showed the backend connection working; the checklist helped make the approval specific.

Responsible Person: Essam